



Prompt It Right: Spring Boot Testing in the AI Era

AI writes the code. Your test suite decides what ships.

A 3-hour deep dive · Devoxx Belgium 2026 · October 5, 09:30



 Development Practices

 Deep Dive 180min

 **BEGINNER**

  67

Prompt It Right: Spring Boot Testing in the AI Era

This 3-hour session teaches Spring Boot testing across the full testing pyramid—unit, sliced, and integration tests with Testcontainers—so you can guide AI to generate useful tests, spot misleading ones, avoid common pitfalls, and improve your test suite with practical patterns and quick wins.

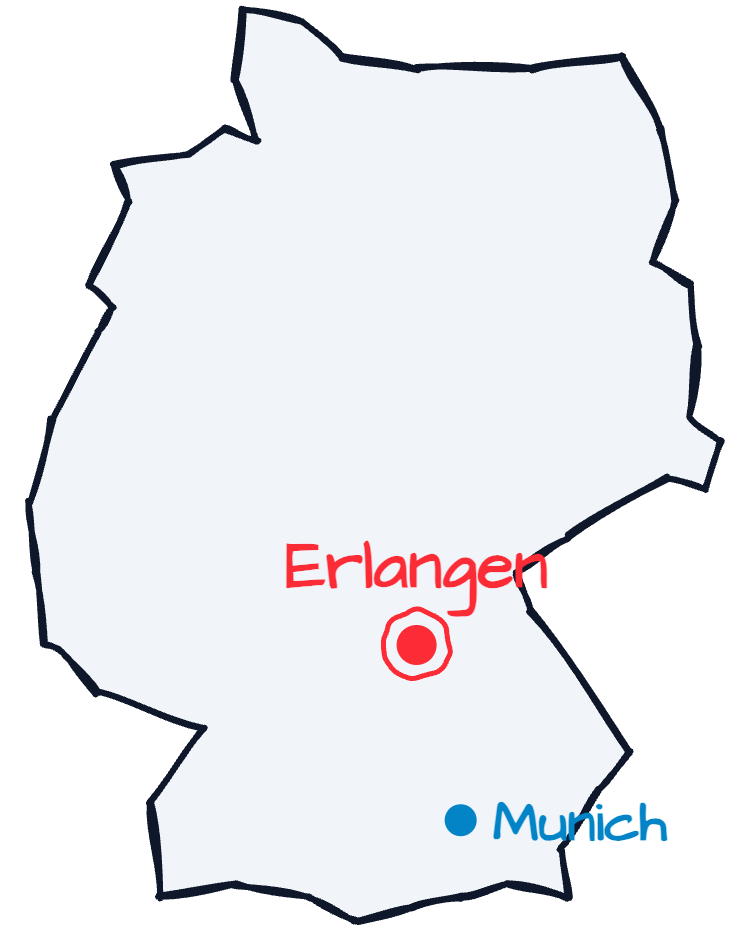
 AI-generated summary — may contain inaccuracies

**Philip Riecks**
PragmaTech GmbH

About Philip

- Software Engineer from Erlangen, Germany 🍺
- Blogging and content creation about testing Java and Spring Boot applications 🌱
- Founder of [PragmaTech GmbH](#) - **Enabling Developers to Frequently Deliver** Software with **More Confidence**
- Using coding agents daily for Spring Boot projects

Where I am from



Erlangen, Bavaria

Help Me Understand You Better

Go to menti.com and enter the code **7108 0067**.

Please answer all **ten questions** - **anonymously**. Your answers help me tailor this session to you.

I will give you a few minutes before we start.



Questions? Three Ways to Ask

1. **Live, during the talk** - raise your hand and interrupt me, it is a beginner-friendly session
2. **Dedicated FAQ** - 10 minutes at the end of each block
3. **Devovx app comments** - scan the QR code, save your question in the comment section of the talk, and I pick up the top ones in the FAQ



2 · Agentic Development and Testing (75 min)

11:15 - 12:30 · problem · skills · best practices · FAQ



Break (30 min)

10:45 - 11:15

1 · Spring Boot Testing in a Nutshell (75 min)

09:30 - 10:45 · unit · slice · integration · Testcontainers · context cache · FAQ

Why Test Software?





Frog told the LLM to write tests for all the code it wrote.

“There,” he said. “Now the model can’t write broken or buggy code.”

“But the model can write broken or buggy tests,” said Toad.

“That is true,” said Frog.

My Northstar for Engineering Excellence

Imagine seeing this pull request on a Friday afternoon:



How **confident** are you to merge this major Spring Boot upgrade and **deploy** it to production once the pipeline turns green?

**Good tests don't just catch bugs -
they give you [fast feedback](#) and
[confident deployments](#).**

Goals for Block 1

A Clear Mental Map

Unit, slice or integration test?

Better Judgment

Review AI-written tests with confidence

The Core Concepts

Reason about test strategy and failures

Ship Fearlessly

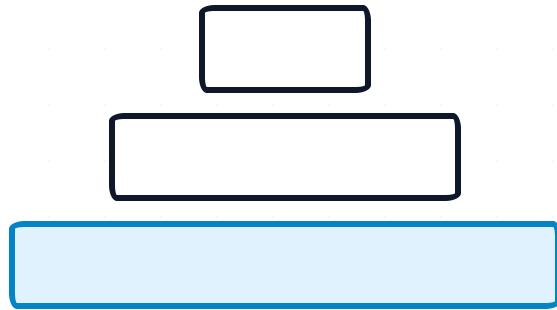
Build confidence into your daily work

01 - SPRING BOOT TESTING IN A NUTSHELL

Know your toolbox

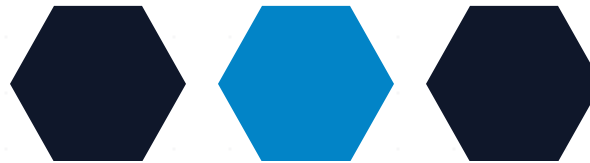
Pick the **cheapest test that proves
the behavior.**

Pyramid, Honeycomb, Trophy: It Depends



Pyramid

one deployable,
layered



Honeycomb

many small
services

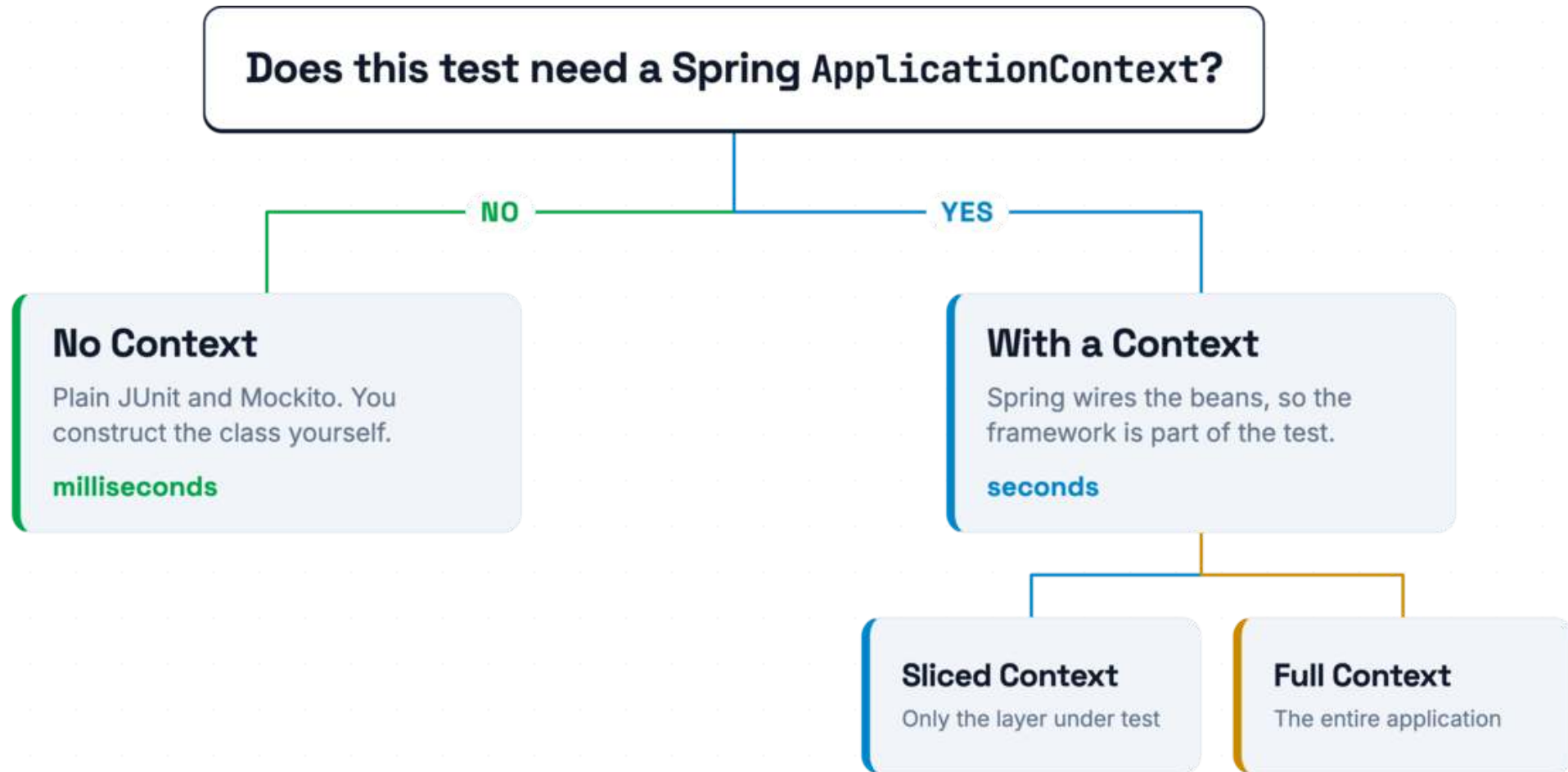


Trophy

integration-
heavy

I don't care about the name. The right shape depends on your project. What matters: **confidence** and **fast feedback**.

Three Ways to Write Tests for Spring Boot Applications



Naming Tests: Keep It Simple

unit test

no Spring context

***Test**

CustomerServiceTest

Everything else

slice, integration, E2E

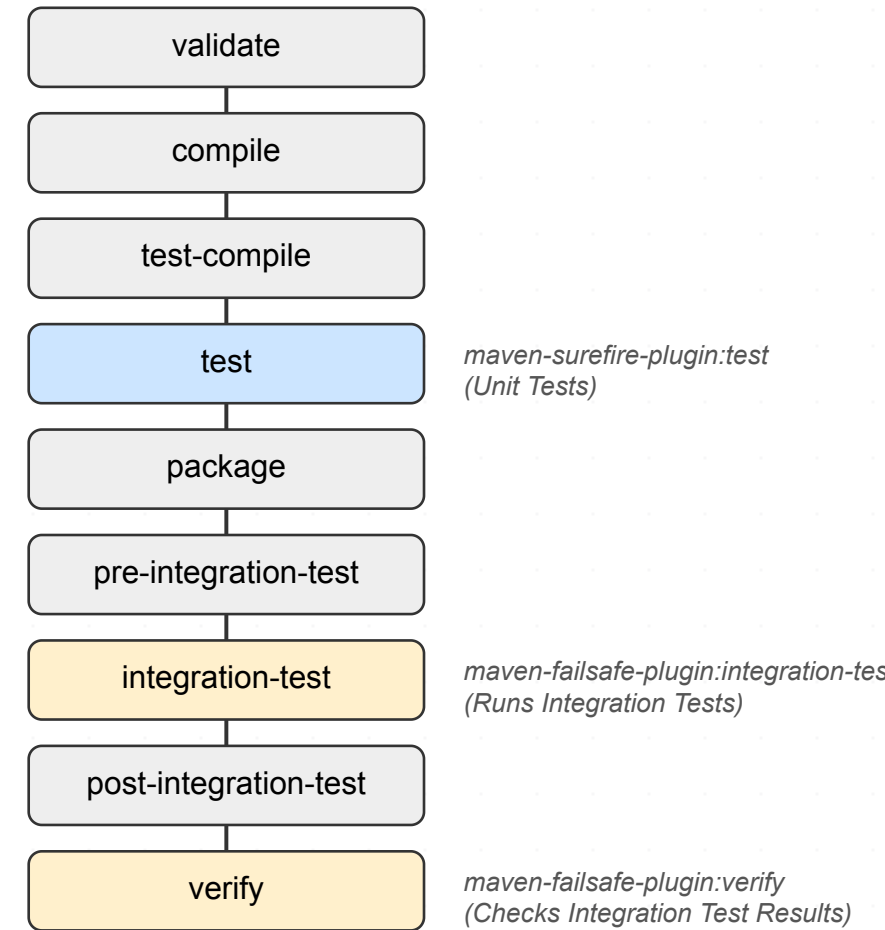
***IT**

CustomerControllerIT

Why Separate the Tests?

- **Different build phases:** Maven Surefire runs `*Test` in the `test` phase, Failsafe runs `*IT` in `integration-test` and `verify`. Gradle: a separate `integrationTest` task
- **Fast feedback first:** unit tests give a verdict in seconds, integration tests run afterwards
- **Configure parallelization differently:** unit tests run fully parallel, integration tests with shared contexts and containers need more care

Running `./mvnw verify`



Legend:

- Surefire Execution Phase
- Failsafe Execution Phases

Our Foundation: Spring Boot Starter Test

- The "Testing Swiss Army Knife"

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-test</artifactId>
4   <scope>test</scope>
5 </dependency>
```

- Batteries-included for testing by transitively including popular testing libraries
- Out-of-the-box dependency management to ensure compatibility



```

1  ./mvnw dependency:tree
2  [INFO] ...
3  [INFO] +- org.springframework.boot:spring-boot-starter-test:jar:4.0.2:test
4  [INFO] | +- org.springframework.boot:spring-boot-test:jar:4.0.2:test
5  [INFO] | +- org.springframework.boot:spring-boot-test-autoconfigure:jar:4.0.2:test
6  [INFO] | +- com.jayway.jsonpath:json-path:jar:2.10.0:test
7  [INFO] | | \- org.slf4j:slf4j-api:jar:2.0.17:compile
8  [INFO] | +- jakarta.xml.bind:jakarta.xml.bind-api:jar:4.0.4:test
9  [INFO] | | \- jakarta.activation:jakarta.activation-api:jar:2.1.4:test
10 [INFO] | +- net.minidev:json-smart:jar:2.6.0:test
11 [INFO] | | \- net.minidev:accessors-smart:jar:2.6.0:test
12 [INFO] | | \- org.ow2.asm:asm:jar:9.7.1:test
13 [INFO] | +- org.assertj:assertj-core:jar:3.27.6:test
14 [INFO] | | \- net.bytebuddy:byte-buddy:jar:1.17.8:test
15 [INFO] | +- org.awaitility:awaitility:jar:4.3.0:test
16 [INFO] | +- org.hamcrest:hamcrest:jar:3.0:test
17 [INFO] | +- org.junit.jupiter:junit-jupiter:jar:6.0.2:test
18 [INFO] | | +- org.junit.jupiter:junit-jupiter-api:jar:6.0.2:test
19 [INFO] | | | +- org.opentest4j:opentest4j:jar:1.3.0:test
20 [INFO] | | | +- org.junit.platform:junit-platform-commons:jar:6.0.2:test
21 [INFO] | | | \- org.apiguardian:apiguardian-api:jar:1.1.2:test
22 [INFO] | | +- org.junit.jupiter:junit-jupiter-params:jar:6.0.2:test
23 [INFO] | | \- org.junit.jupiter:junit-jupiter-engine:jar:6.0.2:test
24 [INFO] | | \- org.junit.platform:junit-platform-engine:jar:6.0.2:test
25 [INFO] | +- org.mockito:mockito-core:jar:5.5.0:test
26 [INFO] | | +- net.bytebuddy:byte-buddy-agent:jar:1.17.8:test
27 [INFO] | | \- org.objenesis:objenesis:jar:3.3:test
28 [INFO] | +- org.mockito:mockito-junit-jupiter:jar:5.5.0:test
29 [INFO] | +- org.skyscreamer:jsonassert:jar:1.5.3:test
30 [INFO] | | \- com.vaadin.external.google:android-json:jar:0.0.20131108.vaadin1:test
31 [INFO] | +- org.springframework:spring-core:jar:7.0.3:compile
32 [INFO] | | +- commons-logging:commons-logging:jar:1.3.5:compile
33 [INFO] | | \- org.jspecify:jspecify:jar:1.0.0:compile
34 [INFO] | +- org.springframework:spring-test:jar:7.0.3:test
35 [INFO] | \- org.xmlunit:xmlunit-core:jar:2.10.4:test

```

What's Inside the Testing Swiss Army Knife?

- **JUnit**: Java's de-facto standard testing framework and foundation.
- **Mockito**: Creating mock objects to simulate dependencies and verify interactions.
- **AssertJ**: Provides fluent, chainable, and readable assertions.
- **Hamcrest**: Offers flexible matchers for creating custom assertions.
- **JSONAssert**: Compares JSON strings with flexible matching options.
- **JsonPath**: Extracts and queries data from JSON similar to XPath.
- **XMLUnit**: Compares and validates XML documents.
- **Awaitility**: Handles asynchronous testing with fluent conditions.

01.1 - UNIT TESTS

No Spring. No excuses.

Unit Testing Java/Spring Boot Applications 101

- **Core Concept:** Test individual components in isolation from dependencies - one unit of work at a time.
- **Confidence Gained:** Fast, high-volume verification that the smallest building blocks behave correctly under various conditions.
- **Pitfall:** Poor class design leads to untestable god classes. Good tests start with good design.
- **Tools:** JUnit, Mockito, AssertJ (or Spock, TestNG, Hamcrest).

Unit Testing has Limits

Consider this sample REST controller, what could we verify with a unit test?

```
1  @RestController
2  @RequestMapping("/api/customers")
3  public class CustomerController {
4
5      private final CustomerService customerService;
6
7      public CustomerController(CustomerService customerService) {
8          this.customerService = customerService;
9      }
10
11     @PostMapping
12     public ResponseEntity<Void> createNewCustomer(@Validated CustomerCreationRequest payload, UriComponentsBuilder uriBuilder) {
13
14         String customerId = customerService.createNewCustomer(payload.firstName());
15
16         UriComponents uriComponents = uriBuilder
17             .path("/api/customers/{id}")
18             .buildAndExpand(customerId);
19
20         return ResponseEntity.created(uriComponents.toUri()).build();
21     }
22 }
```

```

1  @ExtendWith(MockitoExtension.class)
2  class CustomerControllerUnitTests {
3
4      @Mock
5      private CustomerService customerService;
6
7      @InjectMocks
8      private CustomerController customerController;
9
10     @Test
11     void shouldCreateCustomerWhenPayloadRequestIsValid() {
12         when(customerService.createNewCustomer(anyString()))
13             .thenReturn("42");
14
15         ResponseEntity<Void> result = customerController.createNewCustomer(
16             new CustomerCreationRequest("Java", "Duke", "duke@spring.io"),
17             UriComponentsBuilder.newInstance()
18         );
19
20         assertThat(result.getStatusCode().value())
21             .isEqualTo(201);
22         assertThat(result.getHeaders().getLocation().toString())
23             .isEqualTo("/api/customers/42");
24     }
25 }

```

Things a Unit Test Can't Cover: Request Mapping

Request mapping: Does HTTP GET `/api/customers/{id}` actually resolve to our desired method?

```
1  @Test
2  void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4      // ...
5
6      ResponseEntity<Void> result = customerController.createNewCustomer(
7          new CustomerCreationRequest("Java", "Duke", "duke@spring.io"),
8          UriComponentsBuilder.newInstance()
9      );
10 }
```

Things a Unit Test Can't Cover: Validation

Validation: Will an incomplete request body result in a 400 bad request or return an accidental 201?

```
1  @Test
2  void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4      // ...
5
6      ResponseEntity<Void> result = customerController.createNewCustomer(
7          new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),
8          UriComponentsBuilder.newInstance()
9      );
10 }
```

Things a Unit Test Can't Cover: Serialization

Serialization: Are our JSON objects serialized and deserialized correctly?

```
1 ResponseEntity<Void> result = customerController.createNewCustomer(  
2     new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),  
3     UriComponentsBuilder.newInstance());
```

```
1 {  
2     "first-name": "Java",  
3     "last_Name": "Duke",  
4     "email": "duke@spring.io"  
5 }
```

Things a Unit Test Can't Cover: Security

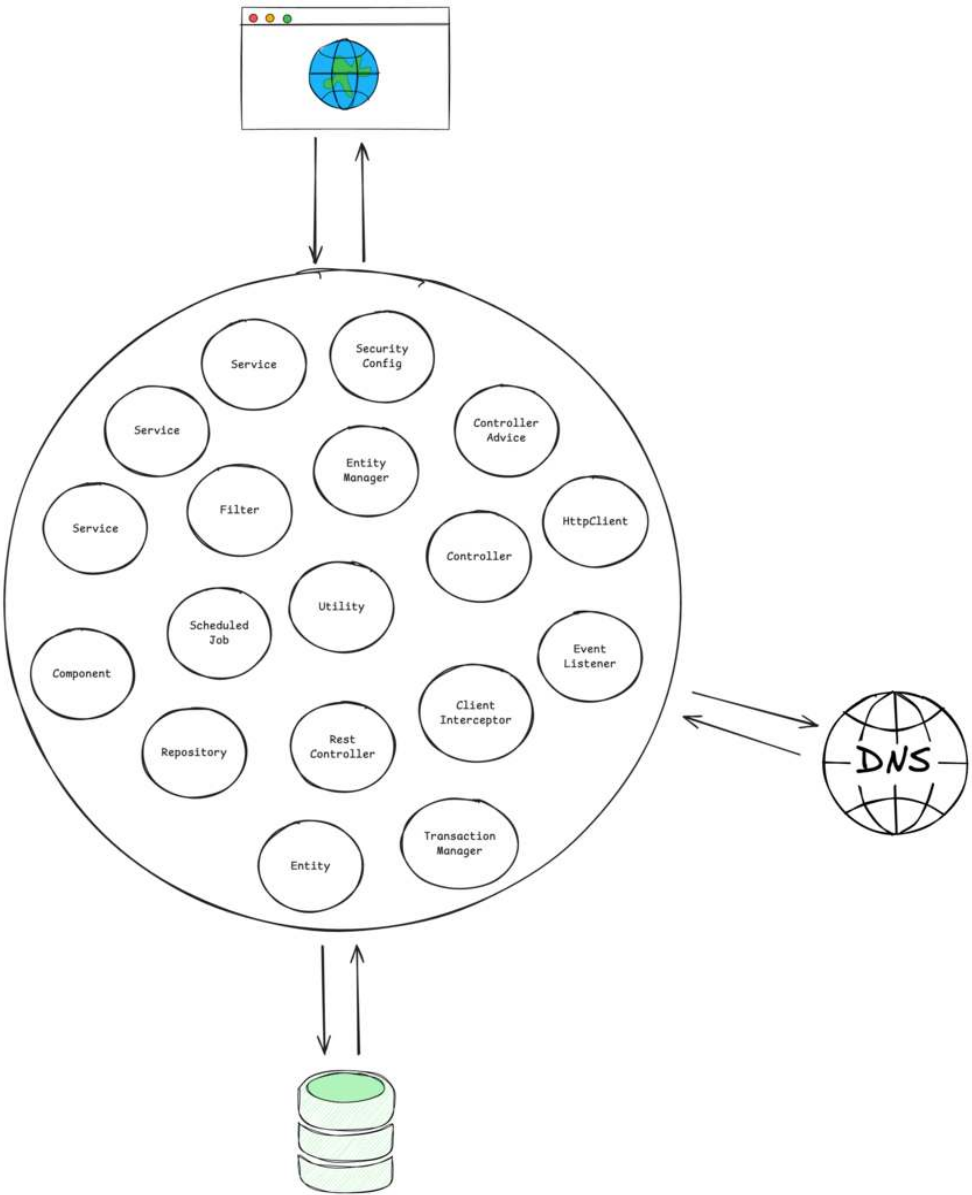
Security: Are our Spring Security configuration and other authorization checks enforced?

```
1  @Test
2  void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4      // ...
5
6      ResponseEntity<Void> result = customerController.createNewCustomer(
7          new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),
8          UriComponentsBuilder.newInstance()
9      );
10 }
```

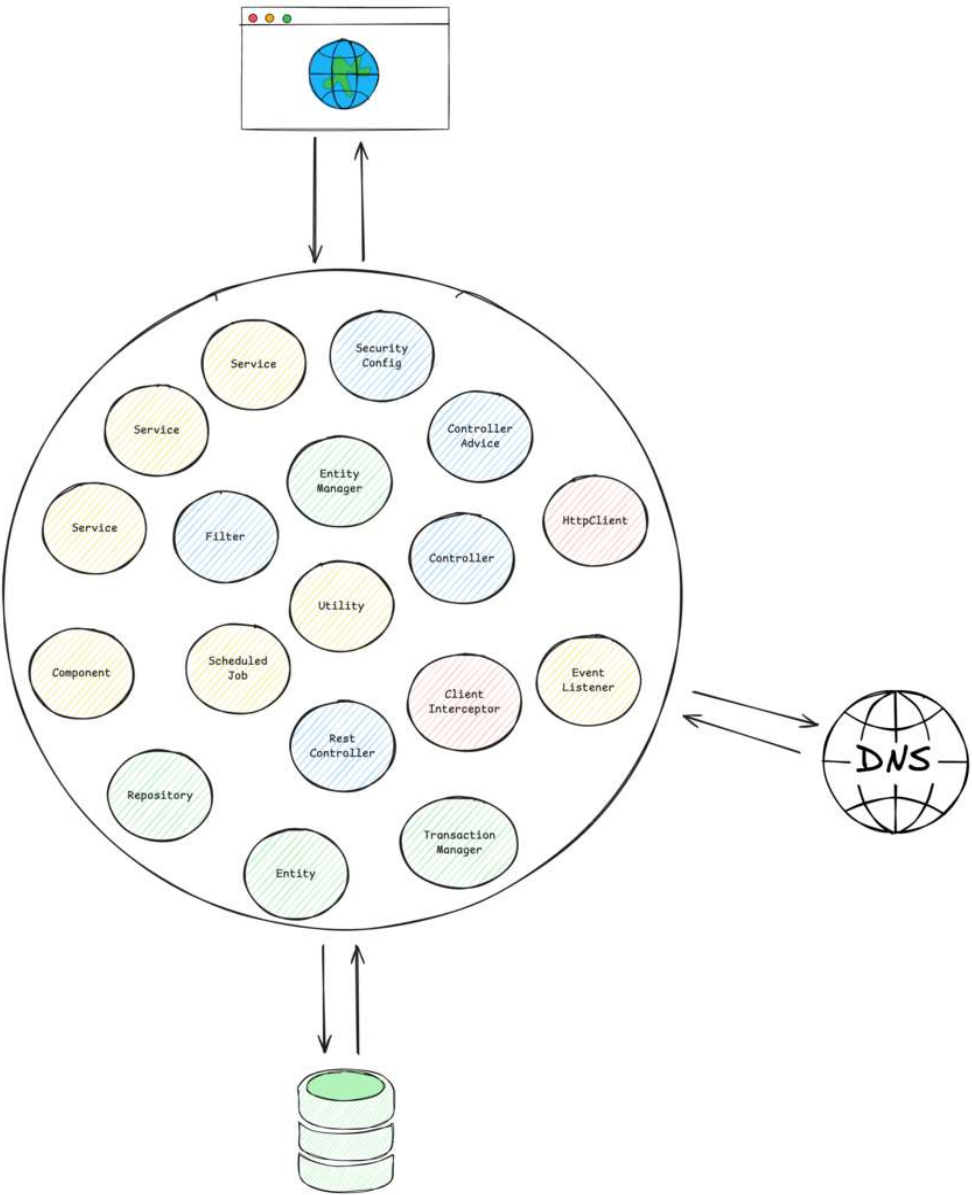
01.2 - SLICE TESTS

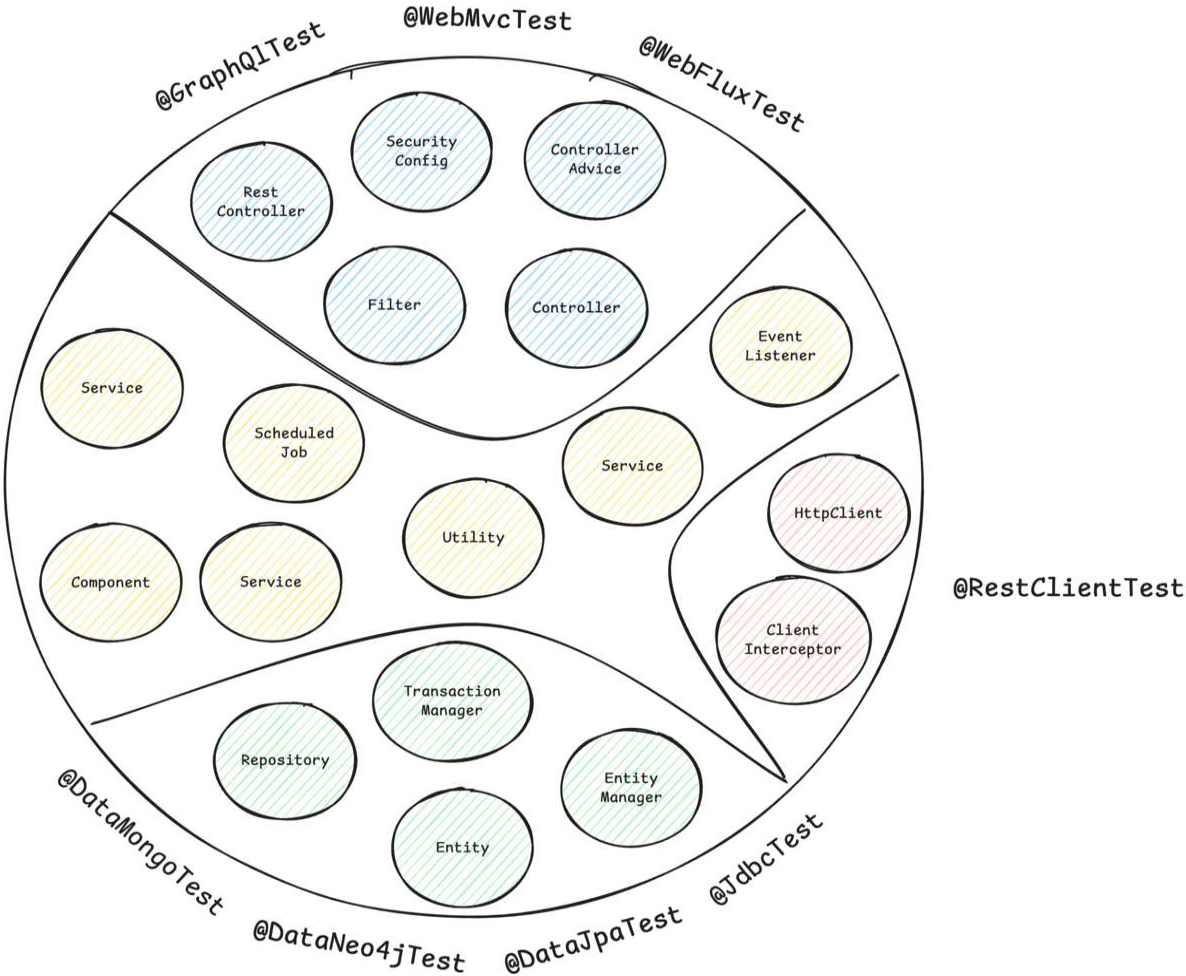
Test one layer

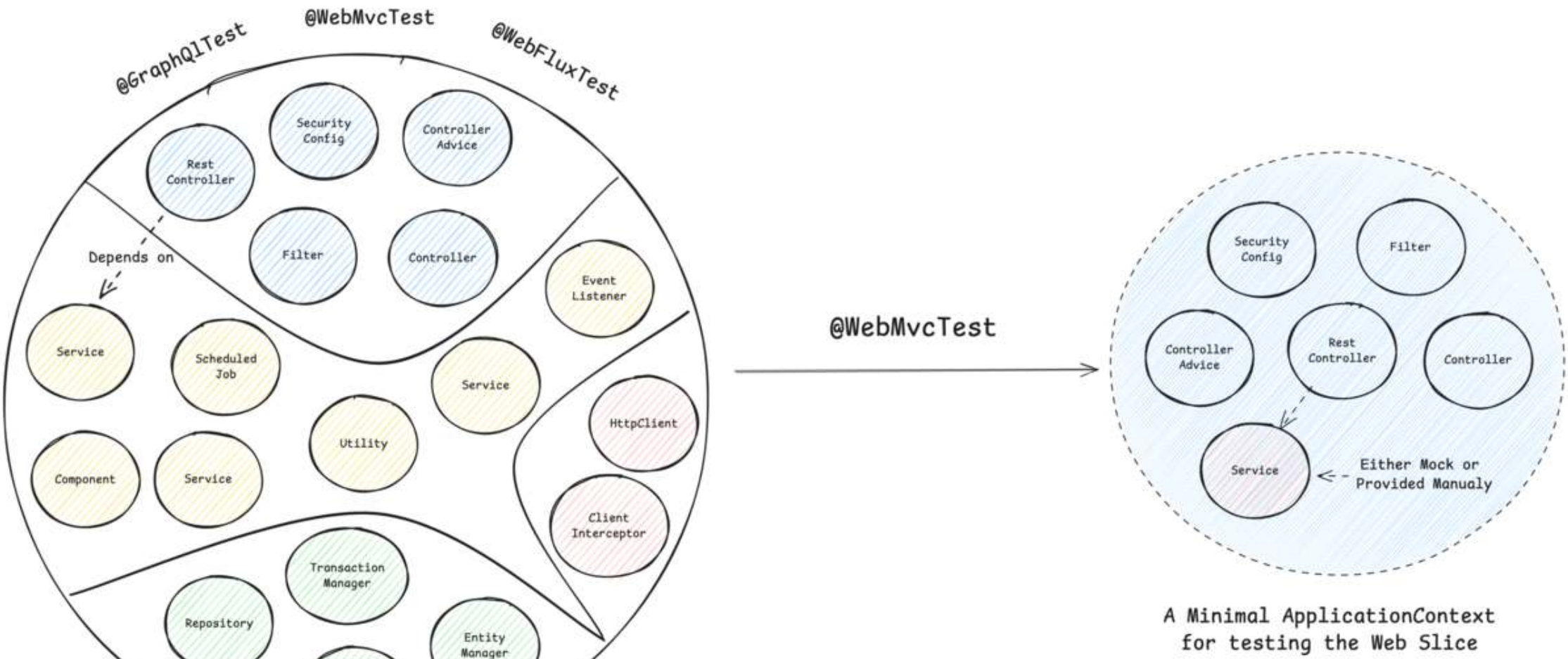
A Typical
Spring ApplicationContext
with different
Spring Beans



A Typical
Spring ApplicationContext







Spring Boot Test Slice Example: @WebMvcTest

```
1  @WebMvcTest(CustomerController.class)
2  @Import(SecurityConfig.class)
3  class CustomerControllerIT {
4
5      @Autowired
6      private MockMvc mockMvc;
7
8      @MockitoBean
9      private CustomerService customerService;
10
11     @Test
12     @WithMockUser
13     void shouldReturnLocationOfNewlyCreatedCustomer() throws Exception {
14         // ...
15     }
16 }
```

Common Test Slices

- `@WebMvcTest` / `@WebFluxTest` - Controller layer
- `@DataJpaTest` / `@JdbcTest` - Persistence layer
- `@JsonTest` - JSON serialization/deserialization
- `@RestClientTest` - RestTemplate testing
- etc.

`@DataCouchbaseTest`
`@DataLdapTest`
`@RestClientTest`
`@JsonTest`
`@JooqTest`
`@WebFluxTest`
`@DataRedisTest`
`@WebMvcTest`
`@DataMongoTest`
`@DataCassandraTest`
`@GraphQLTest`
`@DataNeo4jTest`
`@WriteYourOwn*`
`@DataJpaTest`
`@SqsTest`
`@JdbcTest`
`@DataElasticsearchTest`

Persistence Slice: `@DataJpaTest` on the Real Database

```
1  @DataJpaTest
2  @AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
3  @Import(TestcontainersConfiguration.class)
4  class CustomerRepositoryIT {
5
6      @Autowired
7      private CustomerRepository customerRepository;
8
9      @Test
10     void shouldFindCustomerByEmail() {
11         // ...
12     }
13 }
```

- Do not test against an embedded database when production runs on another one
- Same SQL dialect, same constraints, same behavior - start the real one with Testcontainers

Sliced Testing Spring Boot Applications 101

- **Core Concept:** Test a specific "slice" or layer of your application by loading a minimal, relevant part of the Spring `ApplicationContext`.
- **Confidence Gained:** Helps validate parts of your application where pure unit testing is insufficient, like the web, messaging, or data layer.
- **Prominent Examples:** Web layer (`@WebMvcTest`) and database layer (`@DataJpaTest`)
- **Pitfalls:** Requires careful configuration to ensure only the necessary slice of the context is loaded.
- **Tools:** JUnit, Mockito, Spring Test, Spring Boot, Testcontainers

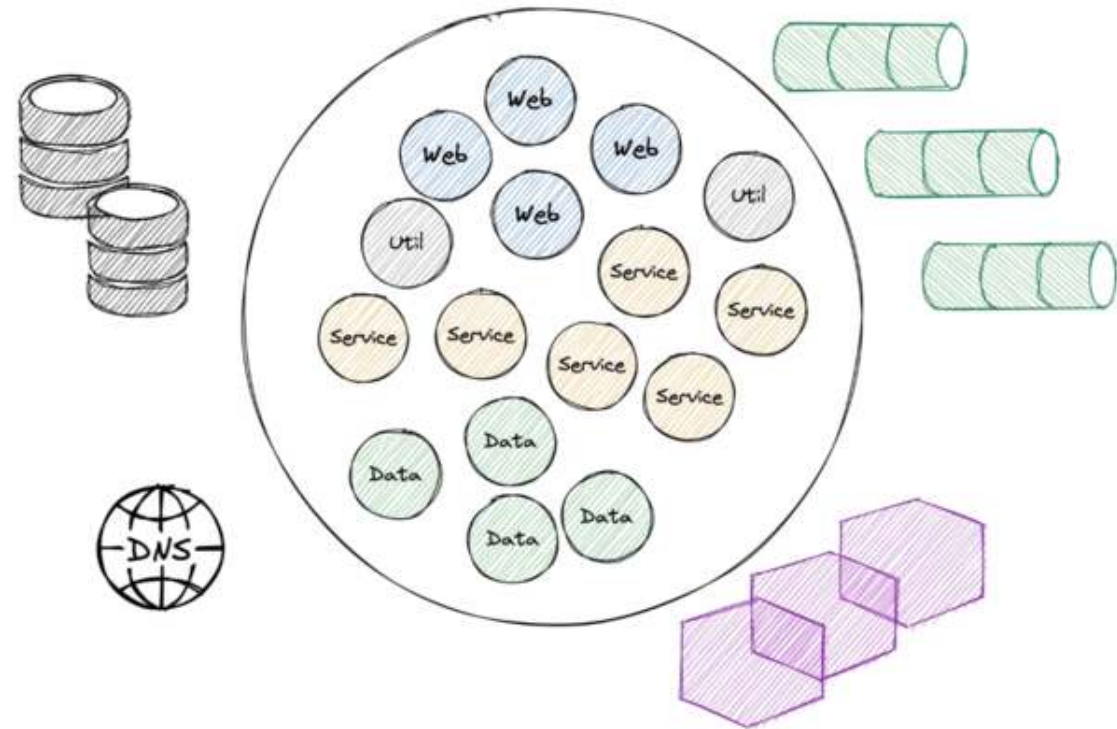
01.3 - INTEGRATION TESTS

The whole app, over HTTP

```
@SpringBootTest
class ApplicationTest {

    @Test
    void contextLoads() {
    }

}
```



Challenges when Starting the Entire `ApplicationContext`

- **Problem #1:** How to ensure surrounding infrastructure (e.g. database, queues, etc.) is present?
- **Problem #2:** How to interact with our application for integration tests?
- **Problem #3:** How to keep our build time at a reasonable duration?

There's Even More...

- **Problem #4:** How to handle HTTP communication from our application to remote services?
- **Problem #5:** How to provide test data and maintain a clean state between tests?
- **Problem #6:** How to handle authentication and security contexts during tests?

Provide External Infrastructure with Testcontainers (Problem #1)

Running infrastructure components (databases, message brokers, etc.) in Docker containers for our tests becomes a breeze with [Testcontainers](#):

```
1 @Container
2 @ServiceConnection
3 static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:16-alpine")
4     .withDatabaseName("testdb")
5     .withUsername("test")
6     .withPassword("test")
7     .withInitScript("init-postgres.sql");
```

This gives us an ephemeral PostgreSQL database for our tests:

```
1 $ docker ps
2 CONTAINER ID   IMAGE                                STATUS      PORTS
3 a958ee2887c6   postgres:16-alpine                 Up 9 seconds  0.0.0.0:32776->5432/tcp
4 ad0f804068dc   testcontainers/ryuk:0.12.0         Up 9 seconds  0.0.0.0:32775->8080/tcp
```

How to Interact with our Application for Integration Tests?

Option #1:

```
1  @SpringBootTest
2  // which is @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.MOCK)
3  @AutoConfigureMockMvc
4  class ApplicationMockWebIT {
5
6      // @LocalServerPort
7      // private int port; <-- this would fail the test, there is no local port occupied
8
9      @Test
10     @WithMockUser
11     void givenCustomersThenReturnListForAuthenticatedUser(@Autowired MockMvc mockMvc) throws Exception {
12         mockMvc
13             .perform(get("/api/customers")
14                 .header(ACCEPT, APPLICATION_JSON))
15             .andExpect(status().is(200))
16             .andExpect(content().contentType(APPLICATION_JSON))
17             .andExpect(jsonPath("$.size()", is(1)));
18     }
19 }
```

How to Interact with our Application for Integration Tests?

Option #2:

```
1  @AutoConfigureWebTestClient // required since Spring Boot 4
2  @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
3  class ApplicationServletContainerIT {
4
5      @Autowired
6      WebTestClient webTestClient;
7
8      @Test
9      void contextLoads() {
10         webTestClient
11             .get()
12             .uri("/api/customers")
13             .header("Authorization", "Basic " + Base64.getEncoder().encodeToString("user:dummy".getBytes()))
14             .exchange()
15             .expectStatus()
16             .isOk();
17     }
18 }
```

Integration Testing Spring Boot Applications 101

- **Core Concept:** Start the entire Spring application context, often on a random local port, and test the application through its external interfaces (e.g., REST API).
- **Confidence Gained:** Validates the integration of all internal components working together as a complete application.
- **Best Practices:** Use `@SpringBootTest` to run the app on a local port.
- **Pitfalls:** Slower to run than unit or sliced tests. Managing the lifecycle of dependent services can be complex.
- **Tools:** JUnit, Mockito, Spring Test, Spring Boot, Testcontainers, WireMock (for mocking external HTTP services), Selenium (for browser-based UI testing)

**... but what about Problem #3:
How to keep our build time at a
reasonable duration?**

01.4 - THE CONTEXT CACHE

The silent speed killer

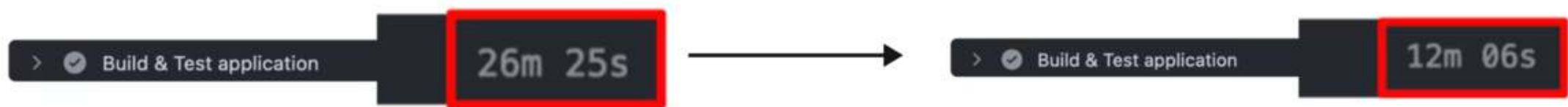
Integration Testing - The Need for Speed

The Problem: Integration tests require a started and initialized Spring `ApplicationContext`, which slows down the build

The Solution: Spring Test `TestContext` caching - stores an already started Spring `ApplicationContext` for later reuse

This feature is part of Spring Test (included in every Spring Boot project via `spring-boot-starter-test`)

Example of speed improvement:



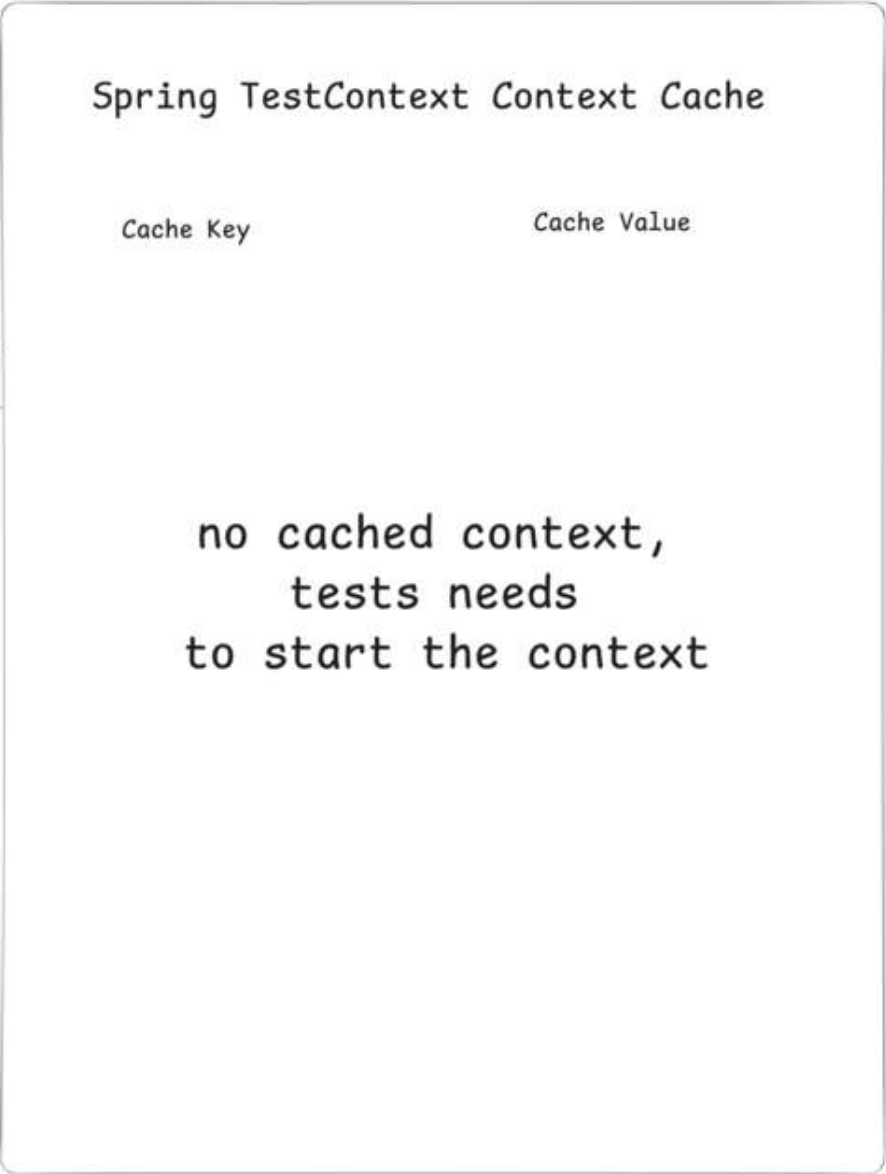
Test run time: 3506ms

OrderIT
requiring context
configuration with the
hash code #327321289

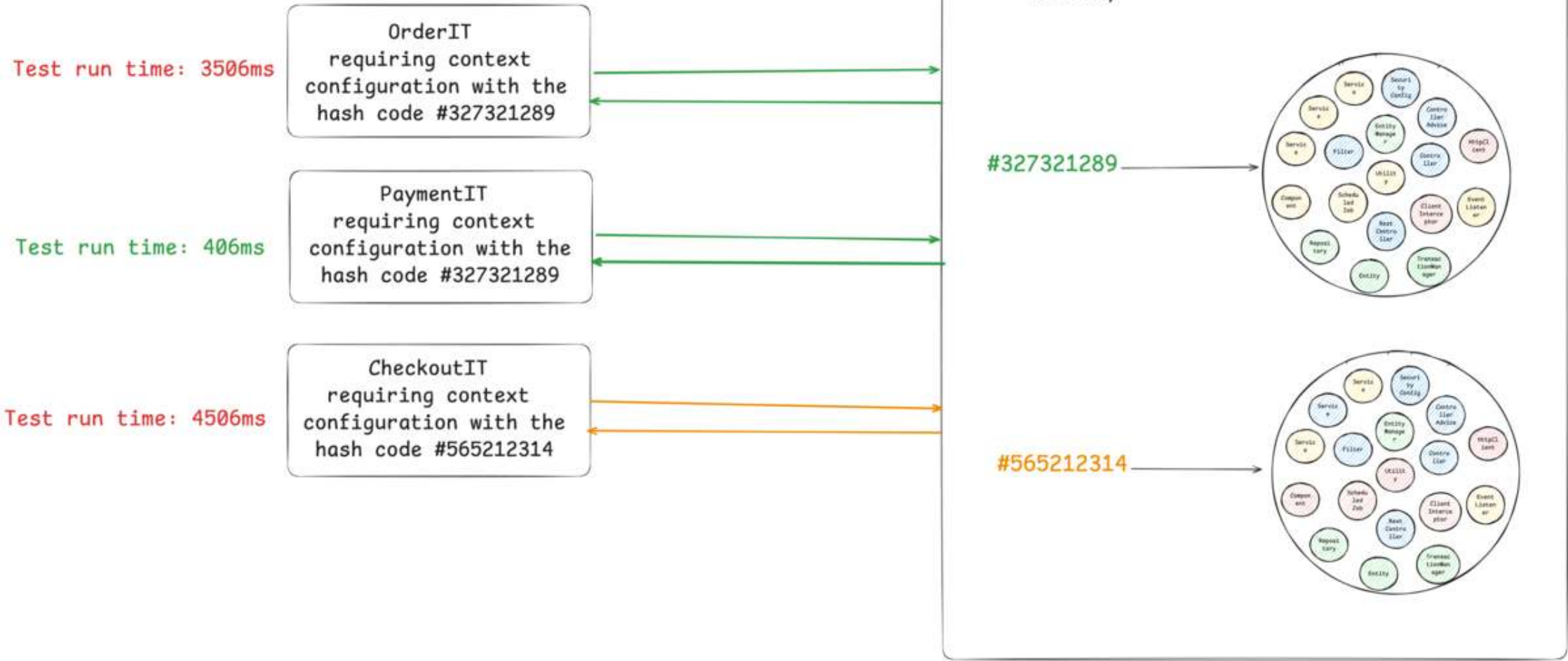
PaymentIT
requiring context
configuration with the
hash code #327321289

CheckoutIT
requiring context
configuration with the
hash code #565212314

Context cache lookup







How the Cache Key is Built

```
1 // DefaultContextCache.java
2 private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
3     Collections.synchronizedMap(new LinkedHashMap<>(32, 0.75f, true));
```

The following information is part of the cache key (`MergedContextConfiguration`):

- activeProfiles (`@ActiveProfiles`)
- contextInitializersClasses (`@ContextConfiguration`)
- propertySourceLocations (`@TestPropertySource`)
- propertySourceProperties (`@TestPropertySource`)
- contextCustomizer (`@MockitoBean`, `@MockBean`, `@DynamicPropertySource`, ...)
- etc.

```
1  Test class
2      |
3      ▼
4  MergedContextConfiguration(
5      testClass, locations, classes,
6      activeProfiles, propertyValues,
7      contextInitializers, contextCustomizers    ← every @MockitoBean lands here
8      ... etc.
9  )
10     |
11     ▼ hashCode() / equals()
12
13  Cache hit? → reuse context ✓
14  Cache miss? → start new context and store it NEW
```


Detect Context Restarts - with Logs

```
<logger name="org.springframework.test.context" level="TRACE" />
```

```
2022-05-05 14:27:38.136 DEBUG 42500 --- [          main] org.springframework.test.context.cache : Spring test
ApplicationContext cache statistics: [DefaultContextCache@68e62b3b size = 1, maxSize = 32, parentContextCount = 0,
hitCount = 11, missCount = 1]
2022-05-05 14:27:38.136 DEBUG 42500 --- [          main] tractDirtiesContextTestExecutionListener : After test class:
context [DefaultTestContext@325bb9a6 testClass = ApplicationIT, testInstance = [null], testMethod = [null], testException
= [null], mergedContextConfiguration = [WebMergedContextConfiguration@1d12b024 testClass = ApplicationIT, locations =
'{}', classes = '{class de.rieckpil.talks.Application}', contextInitializerClasses = '[]', activeProfiles = '{}',
propertySourceLocations = '{}', propertySourceProperties = '{org.springframework.boot.test.context
.SpringBootTestContextBootstrapper=true, server.port=0}', contextCustomizers = set[org.springframework.boot.test.context
.filter.ExcludeFilterContextCustomizer@5215cd9a, org.springframework.boot.test.json
.DuplicateJsonObjectContextCustomizerFactory$DuplicateJsonObjectContextCustomizer@9257031, org.springframework.boot.test
```

Detect Context Restarts - with Tooling



An [open-source Spring Test utility](#) that provides visualization and insights for Spring Test execution, with a focus on Spring context caching statistics.

Overall goal: Identify optimization opportunities in your Spring Test suite to speed up your builds and ship to production faster and with more confidence.

A Common Anti-Pattern: `@DirtiesContext`

Developers tend to consult AI/Stack Overflow for integration test issues and often copy advice from the internet without knowing the implications:

```
1  @SpringBootTest
2  @DirtiesContext
3  // this instructs Spring to remove the context from the cache
4  // and rebuild a new context on every request
5  public abstract class AbstractIntegrationTest {
6
7  }
```

The setup above will **disable** the context caching feature and slow down the builds significantly!

01.5 - SPEED AND QUALITY

Run faster, test better

Test Parallelization

Goal: Reduce build time and get faster feedback

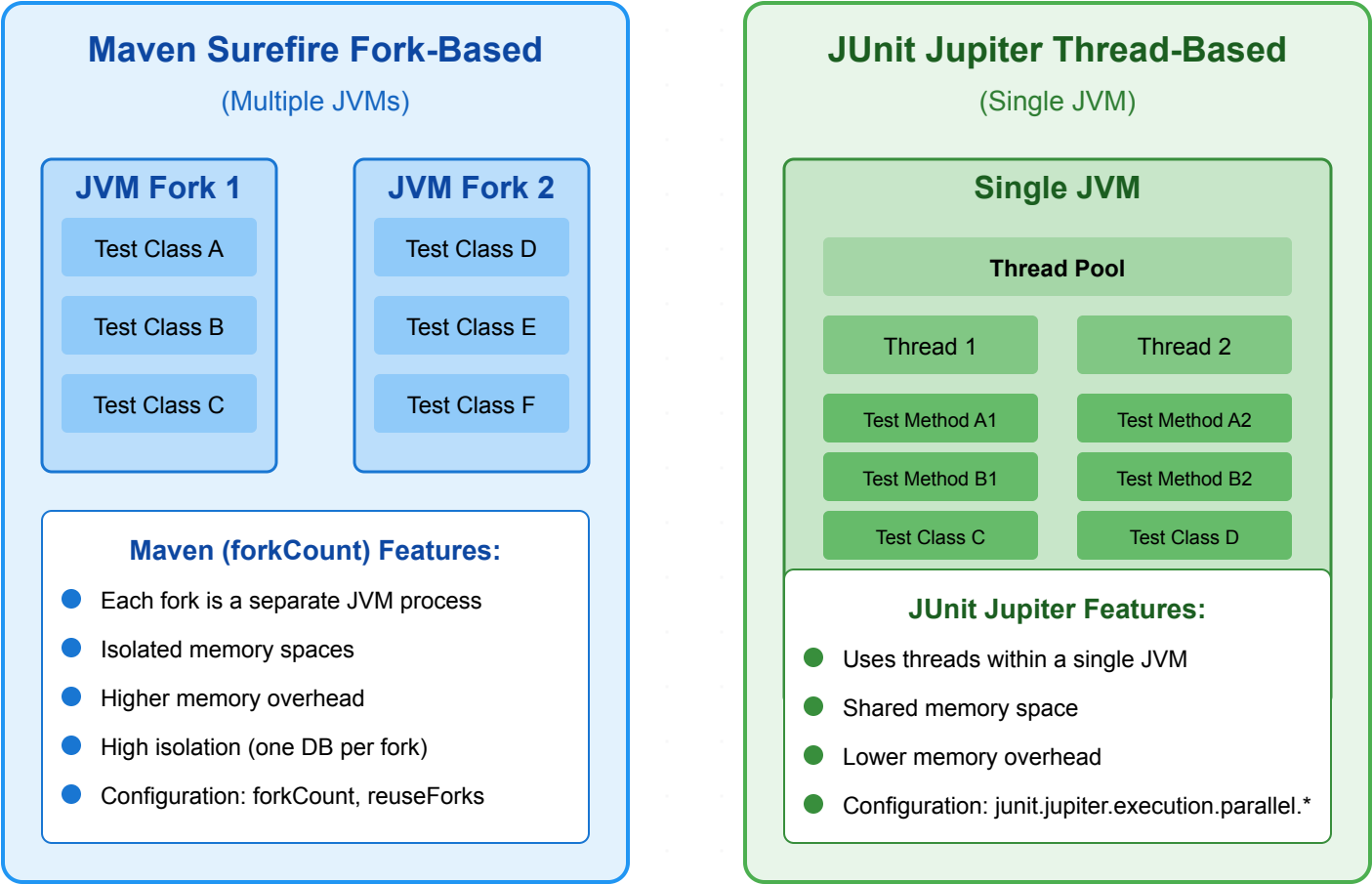
Requirements:

- No shared state
- No dependency between tests and their execution order
- No mutation of global state

Two ways to achieve this:

- Fork a new JVM with Surefire/Failsafe (or for the Gradle test task) and let it run in parallel
- Use JUnit Jupiter's parallelization mode and let it run in the same JVM with multiple threads

Java Test Parallelization Options



Test Parallelization 101

Using Surefire/Failsafe:

```
1 <plugin>
2   <artifactId>maven-surefire-plugin</artifactId>
3   <configuration>
4     <forkCount>1C</forkCount> <!-- 1 JVM per CPU core -->
5   </configuration>
6 </plugin>
```

With JUnit Jupiter:

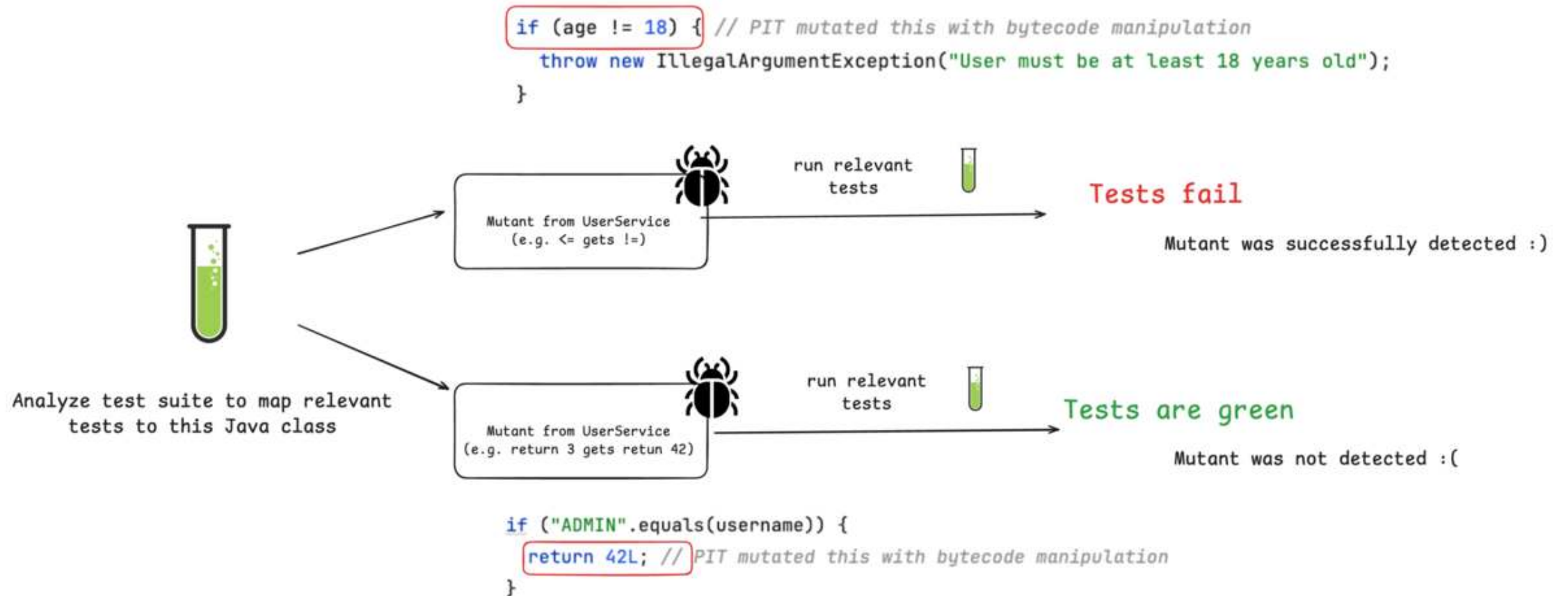
```
1 # src/test/resources/junit-platform.properties
2 junit.jupiter.execution.parallel.enabled = true
3 junit.jupiter.execution.parallel.mode.default = same_thread
4 junit.jupiter.execution.parallel.mode.classes.default = concurrent
```

Let's Challenge Code Coverage

Imagine a set of unit tests for this isolated business logic:

```
1  public Long registerUser(int age, String username) {  
2  
3      if (age <= 18) {  
4          throw new IllegalArgumentException("User must be at least 18 years old");  
5      }  
6  
7      if ("ADMIN".equalsIgnoreCase(username)) {  
8          throw new IllegalArgumentException("Username 'ADMIN' is not allowed");  
9      }  
10  
11     // ...  
12  
13 }
```

Idea: Introduce Regressions to Verify Test Quality



Introducing: Mutation Testing

- Having high code coverage might give you a **false sense of security**
- Mutation Testing with PIT
- Beyond Line Coverage: Traditional tools like JaCoCo show which code runs during tests, but PIT verifies if our tests actually detect when code behaves incorrectly by introducing "**mutations**" to our source code.
- Quality Guarantee: PIT automatically **modifies our code** (changing conditionals, return values, etc.) to ensure our tests fail when they should, **revealing blind spots** in seemingly comprehensive test suites.

E2E Tests in One Slide

- Drive the **running application** through the browser, like a user would
- Keep them **few**: only the critical user journeys
- Use page objects, no locators inside the tests
- Run them in a **separate profile**, not in the fast build
- Tools: Selenium or Playwright, browser in a Testcontainers container

Spring Boot 4 - Testing Support Keeps Improving

- **RestTestClient**: Modern, fluent alternative for the `TestRestTemplate` / `WebTestClient` / `RestAssured`.
- **Context pausing**: Cached test contexts are now automatically paused, eliminating resource conflicts from background processes.
- **JUnit 6**: Drop-in upgrade from JUnit 5 - far smoother than the JUnit 4 → 5 migration.
- **Testcontainers 2.0**: New `testcontainers-` prefix for modules, JUnit 4 support removed.
- **Bean overrides for non-singletons**: `@MockitoBean` and `@TestBean` now work with prototype and custom-scoped beans.

What makes a good test suite: five properties

Fast - the agent loop needs seconds, not minutes

Deterministic - flaky tests teach the agent to retry and ignore

Isolated - parallel runs, random order, no shared state

One reason to fail - a red test points at one defect

A message that locates it - `.as("...")` on every assertion

FAQ 1: Your Questions

Where and how to ask:

1. **Live** - raise your hand and ask right now
2. **Devoxx app comments** - scan the QR code and write your question in the comment section of the talk, I pick the best ones
3. **After the session** - find me in the hallway



BREAK

Back at 11:15

Recap: What We Learned in Block 1

Testing Toolbox

JUnit · Mockito · AssertJ

Spring Test Types

Unit · Slice · Integration

Testcontainers

Real infrastructure

Context Caching

Reuse the context

Parallelization

Isolated tests

Mutation Testing

Coverage lies

02 - THE PROBLEM TODAY

Code is generated. Trust is not.

**Today, most of our code is
generated.**

Testing Was Often an Afterthought

Written last
after the feature

Skimmed fastest
in code review

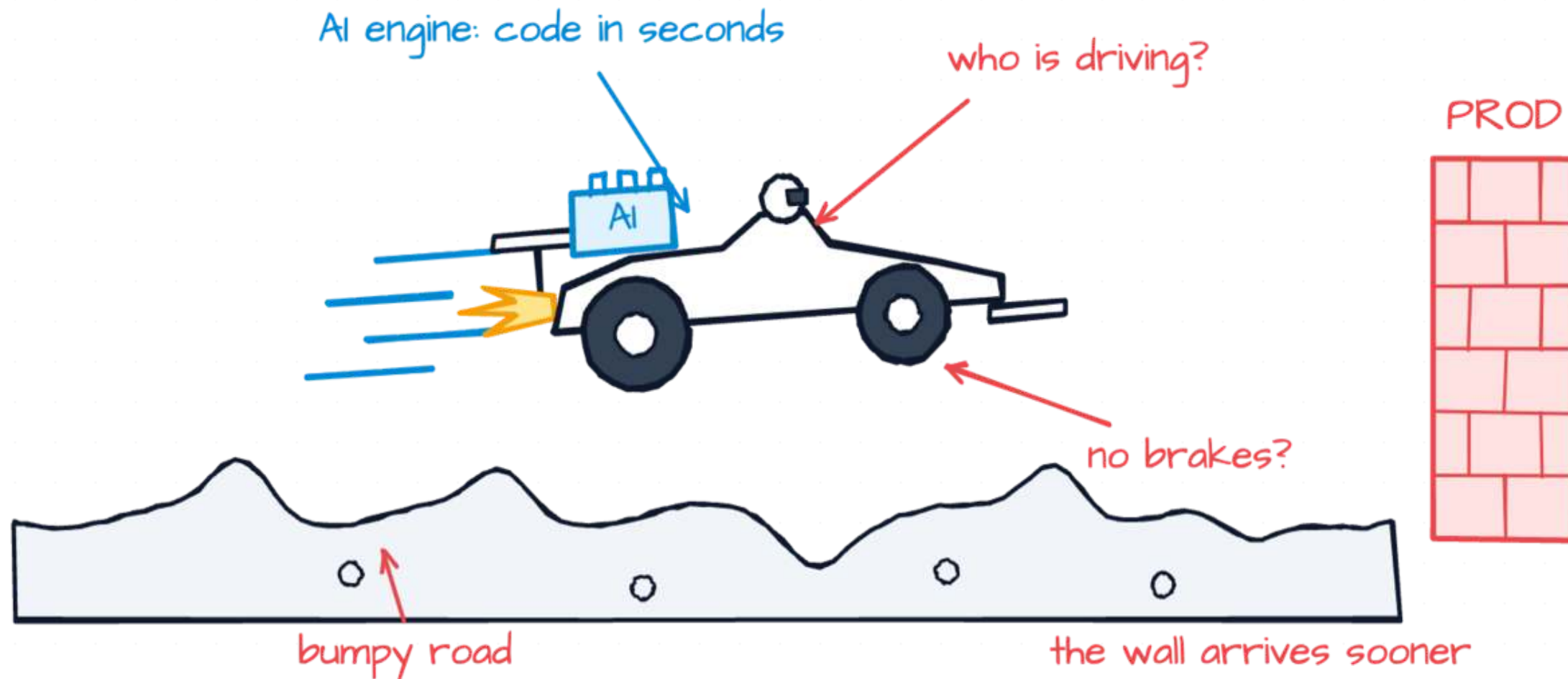
A dashboard
number
nobody reads closely

"Later"
said, not meant

AI won't fix this **by itself.**

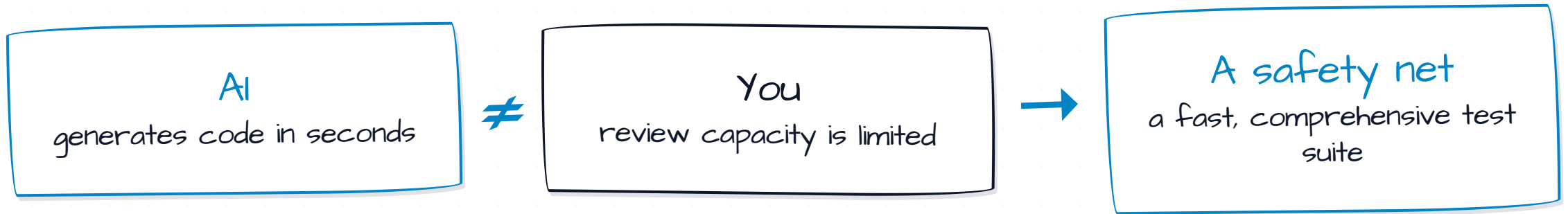
**Somebody handed your team a
Formula 1 engine.**

On a Bumpy Road, Faster Does Not Mean Safer

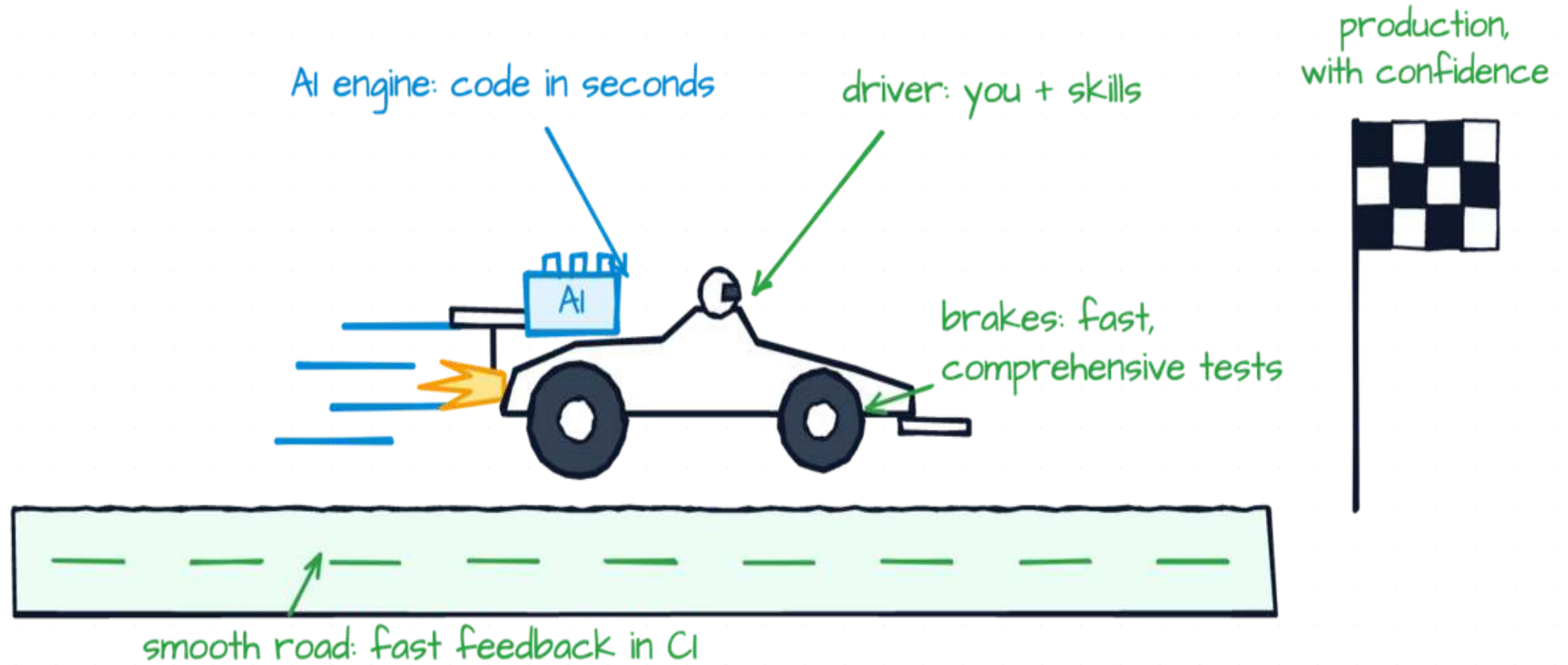


Verification is the new **constraint**.

Code in Seconds, Review Capacity Is Limited



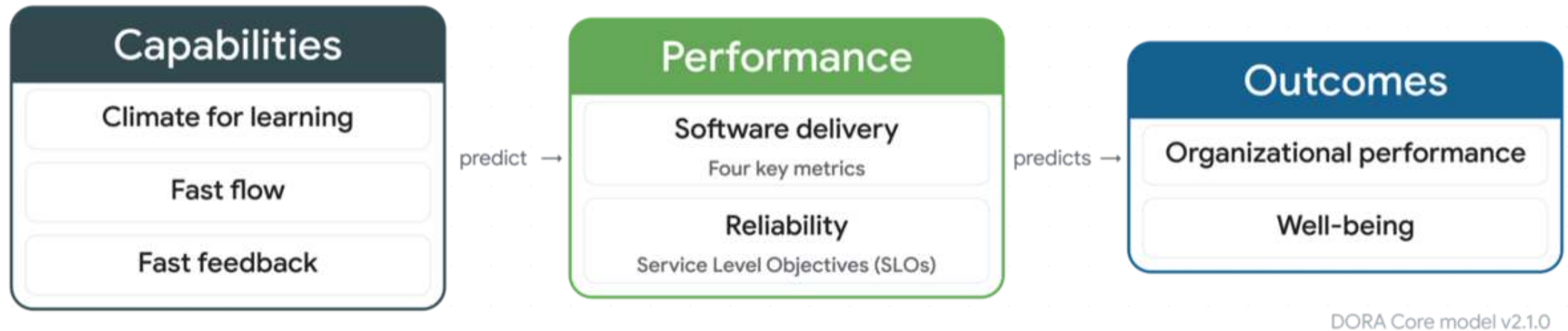
Fix the Brakes, the Road, and the Driver



Backed by the DORA Research

DORA Core Model

dora.dev/research



DORA's core model puts **fast feedback** next to fast flow and a climate for learning.

- These capabilities predict **software delivery performance**
- Delivery performance predicts **organizational performance** and well-being

The Goal: Fast and Comprehensive



03 - MEANINGFUL TESTS

Prompt it right?

The Prompt Everybody Starts With

1 \$ Write meaningful tests. Make no mistakes.

Models get better. But what is meaningful?

Meaningful Is Not Obvious

New developers might not prompt the AI to think about **fast and parallel** tests:

Nobody asked for it	What the agent does
Fast	A new Spring context per test class
Parallel	Shared fields and fixed IDs, tests collide
Deterministic	<code>Instant.now()</code> and <code>Thread.sleep</code>
One reason to fail	Six different assertions in one test
Failure Indicators	A red tests and a nine minute debug

Timing is a big thing. Keep the suite fast.

04 - SKILLS

Testing judgment, written down once



A Skill Is a Folder With One Markdown File

```
1 .claude/skills/unit-testing/
2   └─ SKILL.md                trigger, project knobs, scope, rules, workflow
3   └─ references/
4       └─ testing-standards.md the rule catalog with IDs and severity
5       └─ examples.md          good and bad pairs
```

Sample in this repo: [prompt-it-right/.claude/skills/unit-testing/](https://github.com/prompt-it-right/.claude/skills/unit-testing/).

The Frontmatter Is the Trigger

```
1  ---
2  name: unit-testing
3  description: Write, review or refactor unit tests for Spring Boot
4               classes ... without a Spring context. Use when the user asks for
5               a unit test, mentions JUnit, Mockito or AssertJ ... Not for
6               @SpringBootTest, slice tests or Testcontainers ...
7  ---
8
```

Write the description like a routing rule: when to use it, and when **not** to.

Inside **SKILL.md**

- **Frontmatter**: name and a description written like a routing rule, "Use when ... Not for ..."
- **Adapt this template**: the knobs for your project (assertions, naming, base class, database, build command)
- **Scope**: when the skill is the right tool and what it hands over to
- **Non-negotiables**: the rules that always apply, each with an ID
- **Workflow**: the steps the agent follows, including how to review its own output

Adapt This Template

Knob	Default
Assertion library	AssertJ with <code>.as("...")</code>
Unit test naming	<code><ClassUnderTest>Test</code>
Build command (single class)	<code>./mvnw -q test -Dtest=OrderServiceTest</code>
Build command (unit suite)	<code>./mvnw -q test</code>
Parallel execution	JUnit parallel, random order
Disabled rules	none

The knobs live in one table at the top of `SKILL.md`. The onboarding prompt fills it for your project.



testing-standards.md : Rules With IDs and Severity

```
1 [T7-H]    a failure message on every assertion
2 [T9-H]    no test-class fields, no @BeforeEach
3 [U1-H]    no Spring context in a unit test
4 [U2-H]    time from an injected Clock, never now()
5 [T27-H]   bounded waits, never Thread.sleep
```

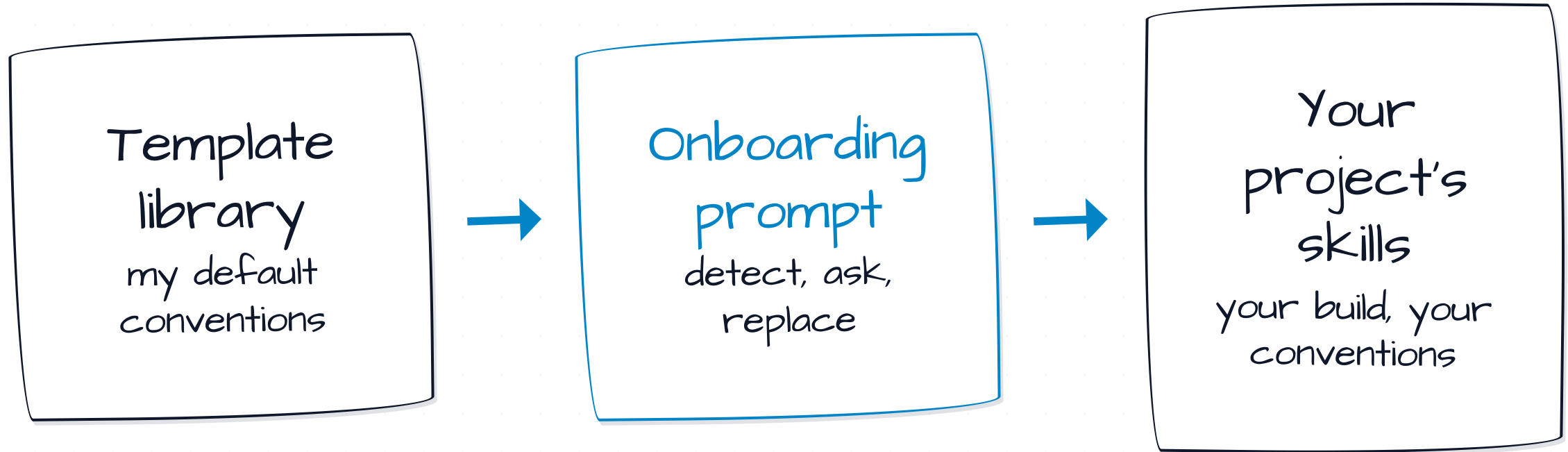
Severity: **C** blocks, **H** must fix, **M** fix or justify, **L** mention only.

The agent reports the rule IDs, so you can see which rule fired and why.

examples.md : Good and Bad Pairs

```
1 // bad: no message, tells you nothing at 3 AM
2 assertThat(order.status()).isEqualTo(PLACED);
3
4 // good: one chain, a message that locates the defect
5 assertThat(order.status())
6   .as("Order is placed after a successful payment")
7   .isEqualTo(PLACED);
```

Models copy examples better than they follow prose.



The Seven Skills

unit-testing

plain classes, no Spring

slice-testing

JPA, JSON, HTTP clients

slice-web-testing

controllers, security

integration-testing

one journey, real HTTP

testcontainers-setup

one container per image

e2e-ui-testing

few browser journeys

test-setup-review

review, read-only

spring-boot-testing

the router, entry point

Demo 1: review a real test suite

Numbers First: PetClinic Suite Review

18

test classes, 76 test
methods

10

cached Spring contexts

0

Docker-free fast phases

What the Review Finds

- 1 [R9-H] no Surefire or Failsafe split, no Docker-free fast phase
- 2 [R16-H] mysql:9.7 declared twice, two containers for one image
- 3 [R16-H] fixed host port 5432, the Postgres test fails when it is taken
- 4 [R17-H] H2 in the tests, MySQL and PostgreSQL in production
- 5 [R13-H] 10 cached contexts, every test class is its own key

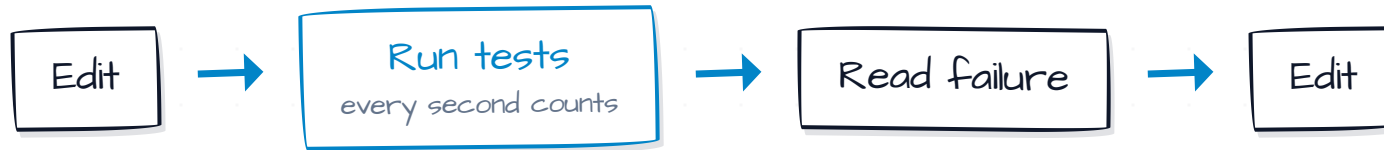
Each line carries a hint, and the file and line behind it.

05 - BEST PRACTICES

Parallelizable and context-cache friendly

Two properties decide your build time: **parallel and **cache-friendly**.**

Why the Agent Loop Cares



A slow suite makes the loop useless, the agent runs one test or none. Every extra Spring context is a full application start.

Must Have: Parallelizable Unit Tests

- No test-class fields, no `@BeforeEach` state: each test builds its own objects
- Time from an injected `Clock`, never `Instant.now()`. No `Thread.sleep`
- Run concurrent and in random order

```
1 # src/test/resources/junit-platform.properties
2 junit.jupiter.execution.parallel.enabled = true
3 junit.jupiter.execution.parallel.mode.default = concurrent
4 junit.jupiter.testclass.order.default = org.junit.jupiter.api.ClassOrderer$Random
5 junit.jupiter.testmethod.order.default = org.junit.jupiter.api.MethodOrderer$Random
```

Must Have: Parallelizable Integration Tests

- **Random business keys** for every piece of test data, own rows per test, no `findAll()` or `count()`
- **Black box over HTTP** on a random port, no shared state in the JVM
- External HTTP stubbed with **WireMock** on a dynamic port, matched on the random values
- `@Execution(CONCURRENT)` on the abstract base class
- Run the suite twice: order dependence shows up before CI

Testcontainers: Use Them Properly

- **One** `static final` container per image, shared by the whole build
- `@ServiceConnection` wires it into Spring Boot, no manual properties
- Pinned image tags, never `latest`. No fixed host ports. An explicit wait strategy
- Same database as production, schema from Flyway or Liquibase, no H2
- Reuse only locally and opt-in, never committed, never on CI. Pre-pull images in CI

Context Caching: One Abstract Base Class

```
1 @SpringBootTest(webEnvironment = RANDOM_PORT)
2 @Import(TestcontainersConfiguration.class)
3 @Execution(ExecutionMode.CONCURRENT)
4 public abstract class AbstractIntegrationTest {
5 }
```

- All integration tests extend it: **one shared context**
- No `@MockitoBean` on application beans, no `@DirtiesContext`
- A context limit (single digit), enforced by the **profiler** in the pipeline

Measure Container Starts With Docker

```
1 docker events --filter event=start \  
2   --format '{{.Actor.Attributes.image}}' | sort | uniq -c
```

Run it in a second terminal while the suite runs. Example output:

```
1      2 mysql:9.7                                <- two containers for one image  
2      1 testcontainers/ryuk:0.12.0
```

- Every image should start **once** per build
- **ryuk** is the Testcontainers cleanup container, ignore it

Measure Context Starts With the Profiler

Every new combination of mocks, properties and profiles is a new Spring context start.

```
1 logging.level.org.springframework.test.context.cache = DEBUG
```

- The cache log shows `size`, `hitCount` and `missCount`
- **Spring Test Profiler** writes the numbers to a JSON file

```
1 jq '.contextsCreated' target/spring-test-profiler/results.json
```

What the Profiler JSON Looks Like

```
1  {
2    "schemaVersion": 1,
3    "profilerVersion": "0.3.0",
4    "totalDurationMs": 1825,
5    "contextsCreated": 2,
6    "contextCacheHits": 0,
7    "contextCacheMisses": 2,
8    "contextCacheHitRatio": 0.0,
9    "springContextCacheSize": 2,
10   "totalContextCreationTimeMs": 1074,
11   "potentialTimeSavingsMs": 273
12 }
```

`target/spring-test-profiler/results.json`, one flat object per run (shortened). Sample project: `prompt-it-right`.

Fail the Build Above the Limit

```
1 - name: Check the Spring context count
2   run: scripts/check-context-count.sh "$MAX_CONTEXTS" \
3       target/spring-test-profiler/results.json
```

```
1 contextsCreated=$(jq -r '.contextsCreated' results.json)
2 [ "$contextsCreated" -gt "$maxContexts" ] && exit 1
```

Limit as a workflow variable, default 9. The HTML report is uploaded as a build artifact.

Demo 2: count the containers

06 - MY DEV SETUP

MCP servers and a local harness

Three Servers, Three Gaps

Server	The gap it closes
Context7	Docs for the framework version you actually use
GitHub	The agent reads its own workflow runs and job logs
Playwright	The agent sees the rendered page, locators are observed

Adding a Server, and Who Gets It

```
1  claude mcp add --transport http <name> <url>
2  claude mcp add <name> -- <command> [args...]
```

Scope	Stored in	Who gets it
local (default)	~/.claude.json	only you, this project
project	.mcp.json	the whole team via git
user	your user config	all your projects



Context7 and GitHub

```
1  claude mcp add --scope user --transport http context7 https://mcp.context7.com/mcp
```

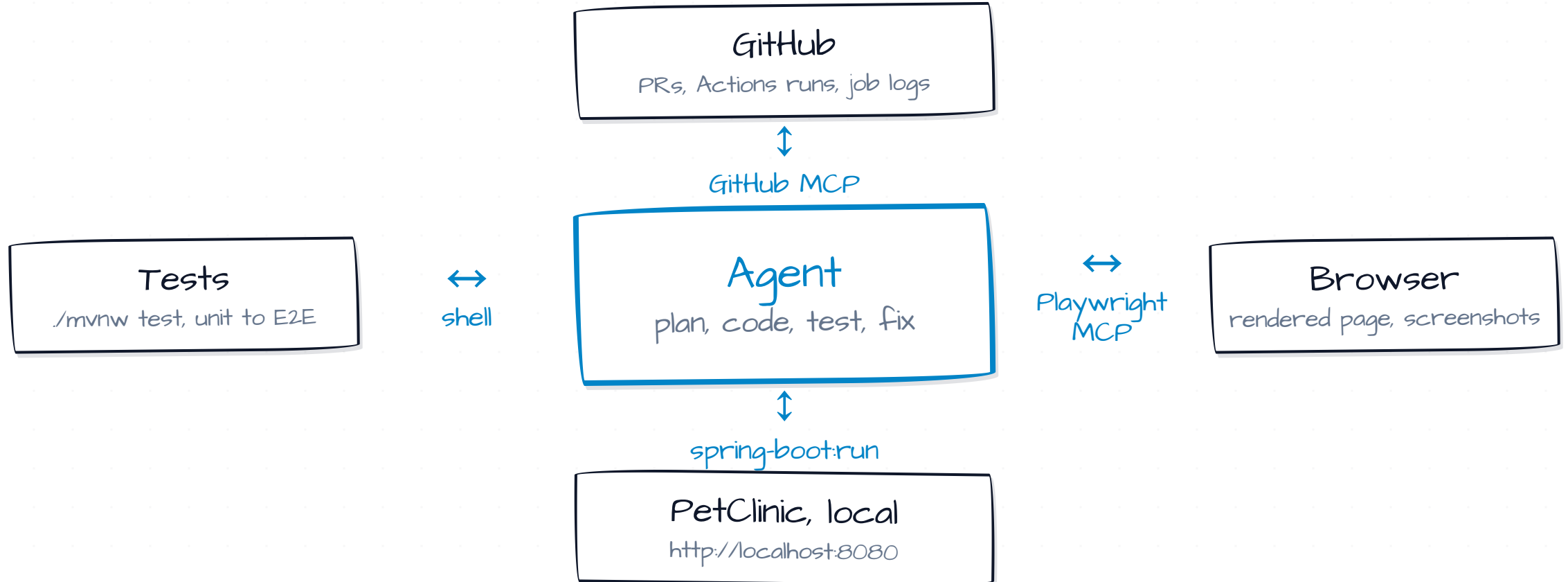
- **Context7**: add `use context7` to the prompt, name the version if it matters
- **GitHub**: Actions toolset to read runs and job logs
- **Read access always on**, write access only with me
- Read-only mode and a token with minimal scopes

Playwright

```
1  claude mcp add playwright npx @playwright/mcp@latest
```

- Works with the accessibility tree, no vision model needed
- `--isolated` keeps the profile in memory, `--storage-state` reuses a saved login
- Visible browser while I watch, a small `slowMo` for debugging
- Only point it at your own applications

My Local Harness: The Agent Closes the Feedback Loop



The Loop in Five Steps

Start PetClinic locally, the agent runs `./mvnw spring-boot:run`

Look through Playwright MCP: open the page, read the rendered DOM and the real element ids

Change code and tests, page objects use the observed locators instead of guessed ones

Run the fast phase, then the browser journey

Push, then read the GitHub Actions run and job logs through GitHub MCP, and fix what failed

Rules I Follow

- **Trust each server:** content it fetches is input, and input can contain instructions
- **Keep the number small:** tool descriptions cost context
- **Never put a token in** `.mcp.json` : use environment variables
- Run `claude mcp list` every few weeks and remove what you do not use

07 - ENGINEERING EXCELLENCE

**Mistakes will
happen. Be ready
to act fast.**

Shift left

tests in the agent loop

CI

fast feedback

CD

small, frequent
releases

Feature flags

deploy is not release

Monitoring

see it in production

Alerting

the right people, fast

Runbooks

know what to do

Fast rollback

undo in minutes

**You will ship a bug. How fast you
recover is what counts.**

Recap: The Last Minutes

Verification

the new constraint

Test Suite

fast and comprehensive

Skills

judgment written down

Parallel and Cached

profiler gate in CI

MCP Servers

few, trusted

Fast Recovery

flags, alerts, rollback

08 - WRAP-UP

What to take home

PetClinic: What the Skill Changed

5 to 1

Spring contexts: team
integration tests vs one
skill-guided journey

0 to 18

`.as()` failure messages
in the unit tests: AI only vs
AI + skill

**H2 to
Postgres**

in the slice tests: AI only
vs AI + skill

Read This Evidence With Care

- **One run per arm.** One sample, not a benchmark
- **No coverage and no mutation score** were measured
- **The prompt named the skill.** In real use the description must trigger it on its own

Five Takeaways

A **fast and comprehensive** suite is the engine for moving fast with agents

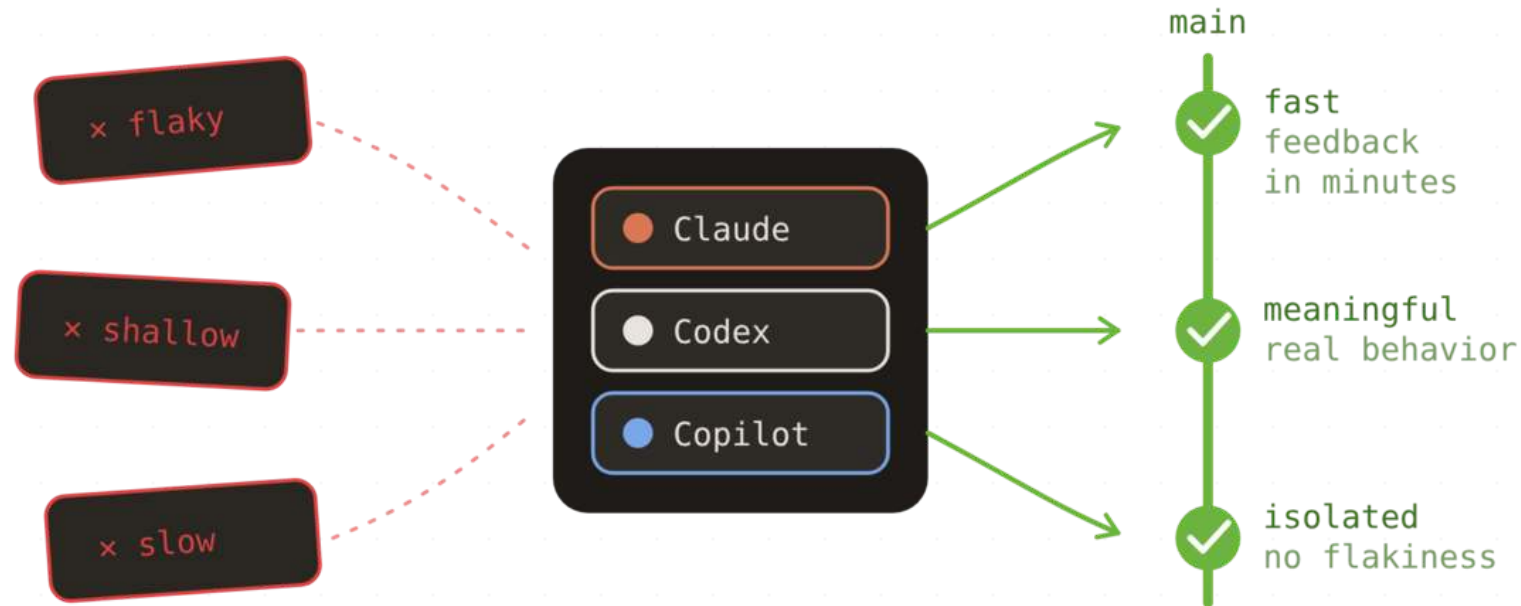
Skills written down once make the agent's result more deterministic

Optimize for **parallel runs** and **context-cache reuse**

Configure **few, trusted MCP servers**

Practice **fast recovery**: feature flags, monitoring, runbooks, rollback

Take It Further: The Online Course



Agentic Testing for Spring Boot

From AI-generated noise to confidence in every commit

Optimizing AI-Written Tests With Skills

Get a **skillset** for fast and comprehensive tests, including a **test strategy** for your project:

```
1 .claude/skills/spring-boot-testing/  
2 |— unit-testing          fast tests without context bloat  
3 |— slice-testing        right-sized Spring context slices  
4 |— slice-web-testing    web layer with @WebMvcTest  
5 |— integration-testing  full-context tests that stay fast  
6 |— testcontainers-setup real infrastructure, one container per image  
7 |— e2e-ui-testing       user journeys against the running app  
8 |— test-setup-review    flags test anti-patterns for you
```

Each skill includes rules, references, best practices and anti-patterns as code and text.

What the Course Covers

Verification is the new constraint: why a trustworthy suite is the base of agentic development

The skill library: the seven skills above, with the rule catalogs behind them

Adopting the skills: one onboarding prompt adapts them to your project, then review your tests and build a feature

My agentic setup: workmode, language server, MCP servers, TDD or not

FAQ 2: Your Questions

Where and how to ask:

1. **Live** - raise your hand and ask right now
2. **Devoxx app comments** - scan the QR code and write your question in the comment section of the talk, I pick the best ones
3. **After the session** - find me in the hallway



Rate this Session

Your feedback helps me improve the talk and helps Devovx.

- Open the **Devovx Belgium app**
- Find "**Prompt It Right**" and leave a rating
- Tell me what worked and what to change

m.devovx.com/events/dvbe26/talks/7006



You can delegate the typing.

You can't delegate the ownership.

You get paged at 3 AM.

**Invest in a test suite that gives
you confidence in every commit.**

Thank you!

Get my template skill structure and **33% off** until the end of Devoxx (Course or Bundle edition):



Philip Riecks · [PragmaTech GmbH](#)

- [LinkedIn](#) (Philip Riecks)
- [Mail](mailto:philip@pragmatech.digital) (philip@pragmatech.digital)

